Advanced Camera Controller System package

TABLE OF CONTENTS

1. Product Overview
2. System Requirements
3. Installation Guide
4. Script Documentation
   - FreeFlyCameraController
   - RTSCameraController
5. Controls Reference
6. Usage Examples
7. Best Practices
8. Troubleshooting
9. Package Contents
10. Changelog
11. License & Support
12. Other Assets by the Publisher

1. PRODUCT OVERVIEW

Advanced Camera Controller System is a comprehensive camera system designed for Unity that provides smooth, intuitive camera control for scene exploration, level design, and showcase purposes. This package includes two fully-featured camera controllers that work seamlessly across both desktop and mobile platforms.

Key Features

- Multiple Control Methods: Mouse, keyboard, and touch support depending upon the Script used

- Smooth Movement: Lerp-based smoothing for professional camera transitions

- Sprint Functionality: Speed boost with Left Shift

- Keyboard Rotation: Full 360° rotation using arrow keys

- Mobile Compatible: Touch input support for iOS and Android

- Easy Integration: Drop-in solution with no external dependencies

- **RTS Camera Support**: Top-down panning with drag controls

- Customizable Parameters: All settings exposed in Unity Inspector

2. SYSTEM REQUIREMENTS

Unity Version

- Unity 2019.4 or higher (all versions up to 2023.3 supported)

Supported Platforms

- Windows (Standalone)
- macOS (Standalone)
- Linux (Standalone)
- iOS
- Android
- WebGL

Input System

- Legacy Input Manager (included with Unity)
- No external plugins or dependencies required

3. INSTALLATION GUIDE

Method 1: Direct Import

1. Download the FreeFlyCameraController package
2. In Unity, go to Assets > Import Package > Custom Package
3. Select the downloaded .unitypackage file
4. Click Import All

Method 2: Manual Installation

1. Extract the package contents to your project's Assets folder
2. Ensure the scripts are in a valid Scripts folder

3. The scripts will automatically appear in Unity's Add Component menu

Initial Setup

1. Select your Main Camera in the Hierarchy
2. Click Add Component in the Inspector
3. Search for "FreeFlyCameraController" or "RTSCameraController"
4. Add the desired component
5. Configure the settings in the Inspector panel
6. Press Play to test

4. SCRIPT DOCUMENTATION

4.1 FreeFlyCameraController

Description

A first-person style free-fly camera that provides smooth movement and rotation controls using both mouse and keyboard input. Ideal for scene exploration, level design, and virtual tours.

Inspector Properties

Property Name	Type	Default Value	Description
Move Speed	float	5f	Base movement speed of the camera in units per second
Sprint Multiplier	float	2f	Speed multiplier applied when holding Left Shift
Rotation Sensitivity	float	2f	Mouse rotation sensitivity for looking around
Keyboard Rotation Speed	float	90f	Keyboard rotation speed in degrees per second
Move Smoothing	float	10f	Movement smoothing factor (higher values = smoother movement)
Rotate Smoothing	float	10f	Rotation smoothing factor (higher values = smoother rotation)
Rotate Left Key	KeyCode	LeftArrow	Key to rotate camera left
Rotate Right Key	KeyCode	RightArrow	Key to rotate camera right
Rotate Up Key	KeyCode	UpArrow	Key to rotate camera up
Rotate Down Key	KeyCode	DownArrow	Key to rotate camera down

Public Methods

Description	Method Name	Parameters
Enables camera control input processing	EnableControls	None
Disables camera control input processing	DisableControls	None
Resets camera to specified position	ResetPosition	Vector3 position
Sets the current movement speed	SetSpeed	float speed

Private Methods

Method Name	Description
Update()	Main update loop
UpdateFreeLook()	Processes all input and applies movement and rotation

Input Processing Logic

Movement:

- Horizontal: A/D or Left/Right Arrow keys

- Vertical: W/S or Up/Down Arrow keys

- Vertical Up/Down: Q (down) and E (up) keys

- Sprint: Left Shift key (multiples speed)

Rotation:

- Mouse: Right mouse button + mouse movement

- Keyboard: Arrow keys (Left/Right for yaw, Up/Down for pitch)

Smoothing:

- Movement uses Vector3.Lerp for position smoothing

- Rotation uses Quaternion.Slerp for rotation smoothing

4.2 RTSCameraController

Description

A top-down RTS-style camera controller that provides panning functionality with drag controls. Perfect for strategy games, map exploration, and top-down viewing. Supports both desktop and mobile platforms.

Inspector Properties

Property Name	Type	Default Value	Description
Pan Speed	float	10f	Speed of camera panning movement
Pan Limit Min	Vector2	(-50, -50)	Minimum X and Z position limits
Pan Limit Max	Vector2	(50, 50)	Maximum X and Z position limits

Public Methods

Method Name	Parameters	Description
SetPanLimits	Vector2 min, Vector2 max	Sets custom pan limits programmatically
ResetCamera	Vector3 position	Resets camera to specified position
EnablePanning	bool enable	Enables or disables panning functionality

Private Methods

Method Name	Description
Update()	Main update loop, processes input based on platform
HandleMouseInput()	Processes desktop mouse drag input
HandleTouchInput()	Processes mobile touch input
PanCamera()	Calculates and applies camera movement

Input Processing Logic

Desktop:

- Left Mouse Button + Drag: Pan camera
- Mouse position delta determines pan direction and speed

Mobile:

- One-finger touch + Drag: Pan camera
- Touch position delta determines pan direction and speed

Limiting:

- Camera position is clamped using Pan Limit Min and Max
- Only X and Z axes are affected (Y position remains constant)

5. CONTROLS REFERENCE

FreeFlyCameraController Controls

Public references for the control keys are given in the inspector to choose the desired keys for controls here but the default are as given in below table,

Control Type	Input	Action
Movement	W	Move Forward
Movement	S	Move Backward
Movement	A	Move Left
Movement	D	Move Right
Movement	Q	Move Down
Movement	E	Move Up
Sprint	Left Shift	Increase movement speed
Rotation	Right Mouse Button + Mouse	Look around
Rotation	Left Arrow	Rotate Left
Rotation	Right Arrow	Rotate Right
Rotation	Up Arrow	Rotate Up
Rotation	Down Arrow	Rotate Down

RTSCameraController Controls

Platform	Input	Action
Desktop	Left Mouse Button + Drag	Pan camera
Mobile	One Finger Touch + Drag	Pan camera

6. USAGE EXAMPLES

Example 1: Basic FreeFly Camera Setup

```
  csharp
using UnityEngine;

public class CameraSetup : MonoBehaviour
{
    private FreeFlyCameraController cameraController;
    void Start()
    {
        // Get the camera controller component
        cameraController = GetComponent<FreeFlyCameraController>();

        // Customize movement settings
        cameraController.moveSpeed = 8f;
        cameraController.sprintMultiplier = 3f;
        cameraController.rotationSensitivity = 2.5f;

        // Customize smoothing
        cameraController.moveSmoothing = 15f;
        cameraController.rotateSmoothing = 12f;
    }
}
```

Example 2: RTS Camera with Custom Limits

```
  csharp
using UnityEngine;

public class RTSGameManager : MonoBehaviour
{
   private RTSCameraController rtsCamera;

   void Start()
   {
      rtsCamera = GetComponent<RTSCameraController>();

      // Set custom pan limits based on map size
      Vector2 mapMin = new Vector2(-100f, -100f);
      Vector2 mapMax = new Vector2(100f, 100f);
      rtsCamera.SetPanLimits(mapMin, mapMax);

      // Adjust pan speed
      rtsCamera.panSpeed = 15f;
   }
}
```

Example 3: Toggle Camera Controls

```
  csharp
using UnityEngine;


public class CameraToggle : MonoBehaviour
{
    private FreeFlyCameraController cameraController;
    private bool controlsEnabled = true;


    void Start()
    {
        cameraController = GetComponent<FreeFlyCameraController>();
    }
    void Update()
    {
        // Toggle controls with Space key
        if (Input.GetKeyDown(KeyCode.Space))
        {
            controlsEnabled = !controlsEnabled;
            if (controlsEnabled)
                cameraController.EnableControls();
            else
                cameraController.DisableControls();
        }
    }
}
```

Example 4: Multi-Camera Setup

```
csharp
using UnityEngine;

public class MultiCameraManager : MonoBehaviour
{
    public Camera freeFlyCamera;
    public Camera rtsCamera;
    private bool isFreeFlyMode = true;

    void Update()
    {
        // Switch between cameras with F1 and F2
        if (Input.GetKeyDown(KeyCode.F1))
        {
            SwitchToFreeFlyMode();
        }
        else if (Input.GetKeyDown(KeyCode.F2))
        {
            SwitchToRTSMode();
        }
    }
```

```
    void SwitchToFreeFlyMode()
    {
        freeFlyCamera.gameObject.SetActive(true);
        rtsCamera.gameObject.SetActive(false);
        isFreeFlyMode = true;
    }

    void SwitchToRTSMode()
    {
        freeFlyCamera.gameObject.SetActive(false);
        rtsCamera.gameObject.SetActive(true);
        isFreeFlyMode = false;
    }
}
```

7. BEST PRACTICES

Performance Optimization

1. **Disable When Not in Use**
   - Set `enabled = false` when camera controls aren't needed
   - Use `EnableControls()` and `DisableControls()` methods

2. **Adjust Smoothing Values**
   - Lower values for more responsive controls (gameplay)
   - Higher values for cinematic smoothness (cutscenes)

3. **Update Frequency**
   - Use `Update()` for general movement
   - Consider `FixedUpdate()` for physics-based scenes

4. **Layer Management**
   - Place camera on its own layer for optimized culling
   - Set appropriate culling masks for performance

Scene Setup Tips

1. **Camera Tag**
   - Always tag your main camera as "MainCamera"
   - Allows other scripts to reference it easily

2. **Multiple Camera Setup**
   - Use different cameras for different views
   - Toggle active state for seamless switching

3. **Input Handling**
   - Avoid conflicts with other input-consuming scripts
   - Use input priority system if needed

Development Workflow

1. **Start with Defaults**
   - Begin with default settings
   - Adjust parameters incrementally

2. **Test on Target Platform**
   - Test controls on all target platforms
   - Consider mobile touch sensitivity differences

3. Use Debug Visualization

- Add debug gizmos for camera path visualization

- Log input values during development

8. TROUBLESHOOTING

Common Issues and Solutions

Issue	Possible Cause	Solution
Camera doesn't move	Script not enabled	Check if script is enabled in Inspector
Camera doesn't move	Input Manager missing axes	Ensure Horizontal/Vertical axes exist
Mouse rotation not working	Right mouse button not pressed	Press and hold right mouse button
Keyboard rotation not working	Arrow keys captured by other scripts	Check for input conflicts
Mobile touch not working	Touch input disabled	Enable touch in Player Settings
Mobile touch not working	No touch support on platform	Verify platform supports touch
Camera jerky movement	Smoothing values too low	Increase Move Smoothing value
Camera overshoots target	Smoothing values too high	Decrease Rotate Smoothing value
RTS camera goes out of bounds	Pan limits not set	Configure Pan Limit Min and Max
FreeFly camera stuck	Rotation values NaN	Reset transform rotation to (0, 0, 0)

Input Manager Configuration

Ensure the following Input Axes are configured in Edit > Project Settings > Input Manager:

Horizontal Axis:

- Name: Horizontal
- Positive Button: d
- Negative Button: a
- Type: Key or Mouse Button

Vertical Axis:

- Name: Vertical
- Positive Button: w
- Negative Button: s
- Type: Key or Mouse Button

Mouse X:

- Name: Mouse X
- Type: Mouse Movement
- Axis: X axis

Mouse Y:

- Name: Mouse Y
- Type: Mouse Movement
- Axis: Y ax

Platform-Specific Issues

WebGL:

- Ensure cursor locking is supported
- Check for input permission prompts

Mobile:

- Verify touch input is enabled in Player Settings
- Test on actual device (emulator may not support touch)

Linux:

- Check for specific input device compatibility
- Ensure mouse buttons are detected correctly

9. PACKAGE CONTENTS

File Structure

FreeFlyCameraController/

|

- Scripts/

| => FreeFlyCameraController.cs

| => RTSCameraController.cs

|

- Documentation/

| => README.txt

| => UserGuide.pdf

|

- Examples/

| => FreeFlyDemoScene.unity

| => RTSDemoScene.unity

| => ExampleScripts/

| => CameraSetupExample.cs

| => RTSGameManagerExample.cs

| => CameraToggleExample.cs

|

File Descriptions

File	Description
FreeFlyCameraController.cs	Main free-fly camera controller script
RTSCameraController.cs	RTS camera controller script
README.txt	Quick start guide and overview
Documentation.pdf	Complete documentation PDF
FreeFlyDemoScene.unity	Example scene with free-fly camera setup
RTSDemoScene.unity	Example scene with RTS camera setup
Scripts/	scripts for reference

10. CHANGELOG

Version 1.0.0 (Initial Release)

- Initial public release
- Free Fly Camera with mouse and keyboard support
- RTS Camera with panning support
- Mobile touch support for both controllers
- Smoothing interpolation for professional movement
- Input smoothing for rotation and movement
- Complete documentation and examples

Version 1.1.0 (Planned)

- Cinematic orbit mode
- Zoom functionality
- Camera shake effects
- Cinemachine integration
- Custom input system support

11. LICENSE & SUPPORT

License Information

FreeFly Camera Controller Package

Copyright © 2024 Magnus Works

This asset is provided "as is" without warranty of any kind, express or implied. The author assumes no liability for any damages arising from the use of this software.

Usage Rights:

- ✓ Commercial use allowed
- ✓ Use in multiple projects allowed
- ✓ Modification allowed
- ✗ Redistribution not allowed
- ✗ Resale not allowed

Support Channels

For technical support, feature requests, or bug reports:

- **Email:** magnusworks5@gmail.com

APPENDIX A: QUICK REFERENCE CARD

FreeFlyCameraController

MOVE: W/A/S/D

UP/DOWN: Q/E

SPRINT: Left Shift

LOOK: Right Mouse + Mouse Move

ROTATE: Arrow Keys

PROPERTIES:

- moveSpeed: 5f
- sprintMultiplier: 2f
- rotationSensitivity: 2f
- keyboardRotationSpeed: 90f
- moveSmoothing: 10f
- rotateSmoothing: 10f

RTSCameraController

PAN: Left Click + Drag (Desktop)

PAN: One Finger + Drag (Mobile)

PROPERTIES:

- panSpeed: 10f
- panLimitMin: (-50, -50)
- panLimitMax: (50, 50)

APPENDIX B: KEYBOARD SHORTCUTS REFERENCE

FreeFly Mode

Shortcut	Function
W	Forward
S	Backward
A	Left
D	Right
Q	Down
E	Up
Shift	Sprint
Right Click + Mouse	Look
Left Arrow	Rotate Left
Right Arrow	Rotate Right
Up Arrow	Rotate Up
Down Arrow	Rotate Down

APPENDIX C: FREQUENTLY ASKED QUESTIONS

Q: Can I use both controllers on the same camera?

A: No, they serve different purposes. Choose the one that fits your needs.

Q: Does this work with Cinemachine?

A: Basic functionality works, but advanced integration is planned for v1.1.

Q: Can I customize the input keys?

A: Yes, all keys are exposed in the Inspector for customization.

Q: Does this work in VR?

A: No, this is designed for standard screen-based camera controls.

Q: Can I animate the camera with this script?

A: Yes, you can use the public methods to control camera position and rotation.

Q: Is this compatible with Unity's new Input System?

A: Not directly, but you can modify the script to use the new Input System.

CONTACT INFORMATION

Developer: Magnus Works

Company: Magnus Works

Email: magnusworks5@gmail.com

ACKNOWLEDGMENTS

Special thanks to the Unity community for testing and providing feedback during development. This asset was created with the goal of providing a simple yet powerful camera solution for Unity developers.

Document Version: 1.0.0

Compatible with Unity 2019.4 - 2023.3

Other Assets by the Publisher If you found this asset useful, you may also be interested in:

- Advanced Character Customization System
- Advanced Formation System

Thank you for purchasing the **Advanced Camera Controller System** package!